

Algorithm Analysis and Design

Module 3 — Complete Exam-Oriented Notes

Greedy Strategy • Dynamic Programming • Backtracking

How this document was built

Built strictly on your Module 3 syllabus. Your uploads (Module 4 lecture notes + Kerala Notes PDF) cover **Dynamic Programming, Floyd-Warshall, Backtracking and N-Queens**; those parts are merged and corrected here.

Greedy, Fractional Knapsack, Kruskal, Prim and Dijkstra were NOT in your uploads — I wrote them from standard textbook knowledge (Section A) with fully worked, machine-verified examples. Your blog index confirms these are the Module 3 topics.

Left out on purpose (not in this syllabus): Divide & Conquer / Merge Sort slides (Module 1–2) and Branch & Bound / TSP (Module 4).

No previous question papers were uploaded, so question priority tags are based on what your notes stress (worked examples, repeated problems) and standard university patterns — never a guarantee.

Tag key: **[Very Important]** **[Important]** **[Practice]**

Contents

Section	Topics
A. Greedy Strategy	Control abstraction • Fractional Knapsack • Kruskal • Prim • Dijkstra
B. Dynamic Programming	Control abstraction • Optimality principle • Matrix Chain Multiplication • Floyd-Warshall
C. Backtracking	Control abstraction • N-Queens problem & algorithm
D. Syllabus coverage check	Every topic accounted for
E. Last-minute revision	Definitions, formulas, algorithms, mistakes
F. Question bank	2-mark, 5-mark, 10/15-mark, numericals, programs, diagrams

A. GREEDY STRATEGY

A.1 Greedy Method — Control Abstraction

Basic idea: Build the answer step by step. At each step pick the choice that looks *best right now* (locally optimal) and never undo it. Like giving change: always take the biggest coin that fits.

Definition

Greedy method is an algorithm design technique that builds a solution piece by piece, always choosing the next piece that offers the most immediate benefit, hoping the sequence of local optima gives a global optimum.

- A **feasible solution** is any subset that satisfies the problem constraints; an **optimal solution** is the feasible one that maximises/minimises the **objective function**.
- Choices are **never reconsidered** (no backtracking).
- Works only if the problem has the **greedy-choice property** (a local best choice is part of some global optimum) and **optimal substructure**. There is no general test in advance — correctness must be proved per problem.

Control abstraction (general template)

```
Algorithm Greedy(a, n)
// a[1:n] contains the n inputs
{
    solution := ∅;
    for i := 1 to n do
    {
        x := Select(a);           // pick the best remaining input
        if Feasible(solution, x) then // does adding x keep it valid?
            solution := Union(solution, x);
    }
    return solution;
}
```

Select chooses an input (and removes it from a); **Feasible** checks constraints; **Union** adds it to the solution. Everything problem-specific hides inside these three procedures — that is what makes it a "control" abstraction.

Greedy vs Dynamic Programming (commonly asked)

Feature	Greedy	Dynamic Programming
Decision	One choice per step, never revised	Tries all choices, keeps the best
Decision sequences	Generates only one	Many generated; sub-optimal ones discarded
Correctness test	No general test	Principle of Optimality
Speed	Usually faster, simpler	Slower, uses a table (more memory)
Example	Fractional knapsack, Kruskal, Prim, Dijkstra	Matrix chain, Floyd-Warshall, 0/1 knapsack

A.2 Fractional Knapsack

Problem: A thief has a bag of capacity M (kg) and n items; item i has weight w_i and profit p_i . Items **can be broken** — a fraction x_i ($0 \leq x_i \leq 1$) can be taken. Maximise total profit.

Formula

Maximise $\sum p_i x_i$ subject to $\sum w_i x_i \leq M$, $0 \leq x_i \leq 1$.

p_i = profit of item i , w_i = its weight, x_i = fraction taken, M = capacity.

Greedy rule: take items in **decreasing order of profit/weight ratio** (p_i/w_i) — the "most profit per kg" first.

```

Algorithm GreedyKnapsack(p, w, M, n)
// items sorted so that p[i]/w[i] >= p[i+1]/w[i+1]
{
    for i := 1 to n do x[i] := 0;
    U := M;                                // remaining capacity
    for i := 1 to n do
    {
        if (w[i] > U) then break;
        x[i] := 1; U := U - w[i];        // take whole item
    }
    if (i <= n) then x[i] := U / w[i];    // take fraction of the next item
}

```

Worked example

Given: $n = 3$, $M = 50$; profits $p = (60, 100, 120)$; weights $w = (10, 20, 30)$.

Item	p	w	p/w	Rank
1	60	10	6	1st
2	100	20	5	2nd
3	120	30	4	3rd

1. Take item 1 fully: weight 10 → remaining 40; profit 60.
2. Take item 2 fully: weight 20 → remaining 20; profit 100.
3. Item 3 weighs 30 > 20 remaining → take fraction $x_3 = 20/30 = 2/3$; profit = $120 \times 2/3 = 80$.

Total profit = 60 + 100 + 80 = 240; solution vector $x = (1, 1, 2/3)$.

- **Time complexity:** sorting dominates → $O(n \log n)$; the selection loop is $O(n)$.
- Greedy is **optimal** for fractional knapsack but **not** for 0/1 knapsack: in the example above, 0/1 greedy takes items 1+2 = 160, whereas items 2+3 give 220.

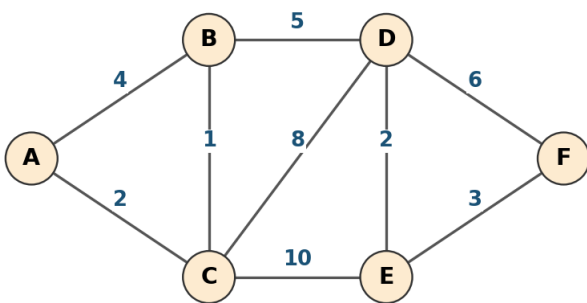
A.3 Minimum Cost Spanning Tree (MST)

Definition

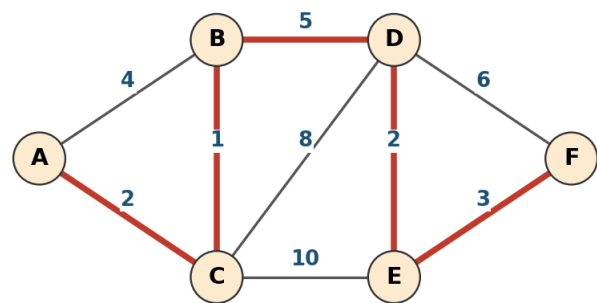
A **spanning tree** of a connected undirected graph $G(V, E)$ is a subgraph that is a tree containing all $|V|$ vertices (so it has exactly $|V|-1$ edges). A **minimum cost spanning tree** is the spanning tree with the smallest total edge weight.

Both algorithms below are greedy and rely on the **cut property**: the lightest edge crossing any cut of the graph belongs to some MST. We use the same graph for both:

Weighted graph used in examples



MST (red edges), total weight = 13



Graph for Kruskal/Prim/Dijkstra examples (left) and its MST (right)

A.3.1 Kruskal's Algorithm

Idea: Sort all edges by weight. Keep picking the cheapest edge, **skip it if it forms a cycle**, until $|V|-1$ edges are chosen. (Cycle check = Union-Find / disjoint sets.)

```

Algorithm Kruskal(G)
{
    A := ∅;
    for each vertex v in V do MAKE-SET(v);
    sort the edges of E into non-decreasing order of weight;
    for each edge (u,v) in sorted order do
        if FIND-SET(u) ≠ FIND-SET(v) then    // different trees → no cycle
            { A := A ∪ {(u,v)};
              UNION(u,v);
            }
    return A;
}

```

Trace (sorted edges: BC1, AC2, DE2, EF3, AB4, BD5, DF6, CD8, CE10):

Step	Edge	Wt	Decision	Reason
1	B-C	1	Accept	joins {B},{C}
2	A-C	2	Accept	joins {A} with {B,C}
3	D-E	2	Accept	joins {D},{E}
4	E-F	3	Accept	joins F with {D,E}
5	A-B	4	Reject	A and B already connected → cycle
6	B-D	5	Accept	joins {A,B,C} with {D,E,F}; now 5 edges = V -1 → stop

MST edges: BC, AC, DE, EF, BD. **Cost** = 1+2+2+3+5 = 13.

Formula

Time complexity = $O(E \log E) = O(E \log V)$. Sorting costs $O(E \log E)$; the union-find operations cost almost $O(E)$. ($\log E \approx \log V$ because $E \leq V^2$.)

A.3.2 Prim's Algorithm

Idea: Start from any vertex. Repeatedly add the **cheapest edge connecting the tree built so far to a vertex not yet in it**. The tree grows like a snowball; it is always connected.

```

Algorithm Prim(G, r)           // r = starting vertex
{
    for each vertex u in V do { key[u] := ∞; parent[u] := NIL; }
    key[r] := 0;
    Q := V;                     // priority queue keyed on key[]
    while Q ≠ ∅ do
    {
        u := EXTRACT-MIN(Q);    // vertex with smallest key
        for each neighbour v of u do
            if (v in Q) and (w(u,v) < key[v]) then
                { parent[v] := u; key[v] := w(u,v); }
    }
}

```

Trace starting from A:

Step	Tree so far	Candidate edges leaving tree	Chosen
1	{A}	AB4, AC2	AC (2)
2	{A,C}	AB4, CB1, CD8, CE10	CB (1)
3	{A,B,C}	BD5, CD8, CE10	BD (5)
4	{A,B,C,D}	DE2, DF6, CE10	DE (2)
5	{A,B,C,D,E}	EF3, DF6	EF (3)

Cost = 2+1+5+2+3 = 13 — same as Kruskal (the MST weight is unique even if the tree is not).

Formula

Time: $O(V^2)$ with adjacency matrix + array; $O(E \log V)$ with adjacency list + binary heap.

Kruskal vs Prim

Feature	Kruskal	Prim
Grows	A forest (many trees merge)	One single tree
Picks	Globally cheapest edge that makes no cycle	Cheapest edge leaving the current tree
Needs	Sorting + Union-Find	Priority queue / key array
Time	$O(E \log V)$	$O(V^2)$ or $O(E \log V)$
Best for	Sparse graphs	Dense graphs
Start vertex	Not needed	Needed
Intermediate result	May be disconnected	Always connected

A.4 Single-Source Shortest Path — Dijkstra's Algorithm**□ Definition**

Given a weighted graph and a source vertex s , Dijkstra's algorithm finds the shortest path from s to every other vertex. It requires **all edge weights ≥ 0** .

Idea: Keep a distance estimate $d[v]$ for each vertex (initially ∞ , $d[s]=0$). Repeatedly pick the unvisited vertex with the smallest d , mark it final, and **relax** its outgoing edges:

Formula

Relaxation: if $d[u] + w(u,v) < d[v]$ then $d[v] = d[u] + w(u,v)$ and $\text{parent}[v] = u$.
 $d[v]$ = current best distance from source to v ; $w(u,v)$ = weight of edge $u \rightarrow v$.

```

Algorithm Dijkstra(G, w, s)
{
  for each vertex v in V do { d[v] := ∞; parent[v] := NIL; }
  d[s] := 0; S := ∅; Q := V;
  while Q ≠ ∅ do
  {
    u := EXTRACT-MIN(Q);           // smallest d[] among unvisited
    S := S ∪ {u};                  // d[u] is now final
    for each neighbour v of u do
      if d[u] + w(u,v) < d[v] then
        { d[v] := d[u] + w(u,v); parent[v] := u; } // relax
  }
}

```

Example (graph above, source A):

Selected	d[A]	d[B]	d[C]	d[D]	d[E]	d[F]
(initial)	0	∞	∞	∞	∞	∞
A (0)	0	4	2	∞	∞	∞
C (2)	0	3 (via C)	2	10	12	∞
B (3)	0	3	2	8 (via B)	12	∞
D (8)	0	3	2	8	10 (via D)	14
E (10)	0	3	2	8	10	13 (via E)
F (13)	0	3	2	8	10	13

Shortest distances from A: B=3 (A→C→B), C=2, D=8 (A→C→B→D), E=10 (...→D→E), F=13 (...→E→F).

Notice A→B direct (4) was improved to 3 via C.

- **Time:** $O(V^2)$ with array; $O(E \log V)$ with binary heap; $O((V \log V) + E)$ with Fibonacci heap.
- **Fails with negative edges** (a vertex already marked final may later get a shorter path). Use Bellman-Ford then.
- Dijkstra solves **single-source**; Floyd-Warshall (Section B) solves **all-pairs**.

MODULE REVISION — Greedy

- **Definitions:** Greedy = locally best choice, never revised. Spanning tree = tree on all V vertices, $V-1$ edges. MST = least total weight.
- **Formulas:** Fractional knapsack: $\max \sum p_i x_i, \sum w_i x_i \leq M$. Relax: $d[v] = \min(d[v], d[u] + w(u, v))$.
- **Algorithms:** GreedyKnapsack $O(n \log n)$ • Kruskal $O(E \log V)$ • Prim $O(V^2)/O(E \log V)$ • Dijkstra $O(V^2)/O(E \log V)$.
- **Differences:** Kruskal (edges + union-find) vs Prim (grow a tree) • Greedy vs DP • Fractional vs 0/1 knapsack.

B. DYNAMIC PROGRAMMING

B.1 Dynamic Programming — Control Abstraction

Basic idea: Break a problem into smaller subproblems, solve each subproblem **only once**, store the answer in a table, and reuse it whenever it is needed again. Simple recursive Fibonacci recomputes the same values exponentially many times; storing them makes it linear.

□ Definition

Dynamic programming is an algorithm design technique for problems whose solution can be viewed as the result of a sequence of decisions, solved by combining stored solutions of overlapping subproblems, using the principle of optimality.

Two properties a problem must have

- **Overlapping subproblems** — the same subproblem is needed again and again (e.g. Fibonacci). Binary search has none, so DP is useless there.
- **Optimal substructure** — an optimal solution contains optimal solutions of its subproblems (e.g. if x lies on the shortest path $u \rightarrow v$, then $u \rightarrow x$ and $x \rightarrow v$ are themselves shortest paths).

Control abstraction — the 4 steps

```
Algorithm DynamicProgramming()
{
  1. Characterize the structure of an optimal solution.
  2. Recursively define the value of an optimal solution.
  3. Compute the value of an optimal solution, typically bottom-up (fill a table).
  4. Construct an optimal solution from the computed information.
}
```

Memoization = top-down recursion that caches results; **tabulation** = bottom-up filling of a table. Both give polynomial time for overlapping subproblems.

B.2 The Principle (Optimality Principle) of Optimality

□ Definition

The principle of optimality states that an optimal sequence of decisions has the property that, whatever the initial state and first decision are, the remaining decisions must constitute an optimal decision sequence with respect to the state resulting from the first decision.

- Equivalent form: **the sub-solutions of an optimal solution are themselves optimal** for their subproblems.
- **Example (satisfies):** shortest path. If $a, x_1, \dots, x_n, b$ is a shortest path from a to b , then the part from x_i to x_j is a shortest path from x_i to x_j .
- **Why it matters:** it is the "litmus test" for DP. Sequences containing a sub-optimal part cannot be optimal, so DP never generates them.
- **Counter-example:** the *longest simple path* does not satisfy it, so DP cannot be applied directly.

Divide & Conquer vs Dynamic Programming

Feature	Divide & Conquer	Dynamic Programming
Subproblems	Independent (disjoint)	Overlapping / interdependent
Split points	Pre-specified (e.g. the middle)	Every possible split tried, best chosen
Approach	Top-down recursion	Bottom-up table (or memoised recursion)
Repeated work	May solve same subproblem many times if they overlap	Solves each subproblem once, stores it
Examples	Merge sort, quick sort, binary search	Matrix chain, Floyd-Warshall, 0/1 knapsack

△ Correction / Flag

Your Kerala notes say D&C; is "less efficient / more time consuming" and DP is "non-recursive". That is an over-generalisation. D&C; is perfectly efficient when subproblems are independent (merge sort is $O(n \log n)$); it only wastes time when subproblems overlap. DP is still *defined* recursively — it is just *evaluated* bottom-up. Write the table above in exams instead.

B.3 Matrix Chain Multiplication (Optimal Matrix Multiplication)

Background: Multiplying a $p \times q$ matrix by a $q \times r$ matrix needs **$p \cdot q \cdot r$ scalar multiplications** and gives a $p \times r$ matrix. Matrices are multiplied only if *columns of first = rows of second*. Multiplication is **associative** but not commutative, so the bracketing changes the cost, not the answer.

Example: $A_1(5 \times 4)$, $A_2(4 \times 6)$, $A_3(6 \times 2)$. $(A_1 A_2) A_3 = 5 \cdot 4 \cdot 6 + 5 \cdot 6 \cdot 2 = 120 + 60 = \mathbf{180}$; $A_1 (A_2 A_3) = 4 \cdot 6 \cdot 2 + 5 \cdot 4 \cdot 2 = 48 + 40 = \mathbf{88}$. Brackets matter!

□ Definition

Matrix-chain multiplication problem: given a chain $A_1 A_2 \dots A_n$ where A_i has dimension $p_{i-1} \times p_i$, fully parenthesize the product so that the number of scalar multiplications is minimum. (We only find the *order*; we do not actually multiply.)

Step 1 – Structure. Split the product $A_{i..j}$ at some k ($i \leq k < j$) into $A_{i..k}$ and $A_{k+1..j}$. Cost = $\text{cost}(A_{i..k}) + \text{cost}(A_{k+1..j}) + \text{cost of multiplying the two results}$. Both sub-chains must themselves be optimal (principle of optimality).

Step 2 – Recurrence**Formula**

$m[i, j] = 0$ if $i = j$

$m[i, j] = \min \text{ over } i \leq k < j \text{ of } \{ m[i, k] + m[k+1, j] + p_{i-1} \cdot p_k \cdot p_j \}$ if $i < j$

$m[i, j]$ = min scalar multiplications for $A_{i..j}$; **k** = split point; **$p_{i-1} p_k p_j$** = cost of multiplying the $(p_{i-1} \times p_k)$ result by the $(p_k \times p_j)$ result. **$s[i, j]$** = the k that gave the minimum.

Step 3 – Algorithm (bottom-up, by increasing chain length)

```
Algorithm Matrix_Chain_Order(p)          // p[0..n], n = p.length - 1
{
  for i := 1 to n do m[i, i] := 0;
  for l := 2 to n do                      // l = chain length
    for i := 1 to n-l+1 do
      {
        j := i + l - 1;
        m[i, j] := ∞;
        for k := i to j-1 do
          {
            q := m[i, k] + m[k+1, j] + p[i-1]*p[k]*p[j];
            if q < m[i, j] then { m[i, j] := q; s[i, j] := k; }
          }
      }
  return m and s;
}
```

Step 4 – Print the optimal bracketing

```
Algorithm Print_Optimal_Parens(s, i, j)    // call with (s, 1, n)
{
  if i = j then print "A", i
  else { print "(";
        Print_Optimal_Parens(s, i, s[i, j]);
        Print_Optimal_Parens(s, s[i, j]+1, j);
        print ")"; }
}
```

△ Correction / Flag

In your lecture slides the symbol “ α ” is used for infinity (∞), and one line reads “ $A_{k+1}A_{i+1}\dots A_j$ ” — it should be $A_{k+1}A_{k+2}\dots A_j$. Also the brute-force recursion takes exponential time because the number of parenthesizations grows like the Catalan numbers.

Formula

Time complexity = **$O(n^3)$** (about $n^2/2$ table entries, each needs up to $n-1$ choices of k). Space = **$O(n^2)$** for tables m and s .

Exam procedure (reusable)

1. Write $p = (p_0, p_1, \dots, p_n)$ from the given dimensions.
2. Fill the diagonal $m[i,i] = 0$.
3. For chain length 2, 3, ..., n compute $m[i,j]$ for every i using all k , writing the split values; note the best k in $s[i,j]$.
4. Answer = $m[1,n]$. Read the bracketing from s using Print_Optimal_Parens.

Worked Example 1 (very likely type): dimensions 5×4 , 4×6 , 6×2 , 2×7

Given: $A_1=5 \times 4$, $A_2=4 \times 6$, $A_3=6 \times 2$, $A_4=2 \times 7 \rightarrow p = (5, 4, 6, 2, 7)$, $n = 4$.

Chain	Computation	$m[i,j]$	$s[i,j]$
A_1A_2	$k=1: 0+0+5 \cdot 4 \cdot 6$	120	1
A_2A_3	$k=2: 0+0+4 \cdot 6 \cdot 2$	48	2
A_3A_4	$k=3: 0+0+6 \cdot 2 \cdot 7$	84	3
$A_{1..3}$	$k=1: m[1,1]+m[2,3]+5 \cdot 4 \cdot 2 = 0+48+40 = \mathbf{88}$; $k=2: m[1,2]+m[3,3]+5 \cdot 6 \cdot 2 = 120+0+60 = 180$	88	1
$A_{2..4}$	$k=2: 0+84+4 \cdot 6 \cdot 7 = 252$; $k=3: m[2,3]+0+4 \cdot 2 \cdot 7 = 48+56 = \mathbf{104}$	104	3
$A_{1..4}$	$k=1: 0+104+5 \cdot 4 \cdot 7 = 244$; $k=2: 120+84+5 \cdot 6 \cdot 7 = 414$; $k=3: m[1,3]+0+5 \cdot 2 \cdot 7 = 88+70 = \mathbf{158}$	158	3

m	1	2	3	4	s	1	2	3	4
1	0	120	88	158	1	-	1	1	3
2		0	48	104	2		-	2	3
3			0	84	3			-	3
4				0	4				-

Reading s : $s[1,4]=3 \rightarrow (A_1A_2A_3)(A_4)$; $s[1,3]=1 \rightarrow A_1(A_2A_3)$. **Optimal order:** $((A_1(A_2A_3))A_4)$ with minimum **158 scalar multiplications**.

Results of the other two examples in your notes (solve them the same way to practise)

Problem	p	Min cost $m[1,n]$	Optimal parenthesization
6 matrices: 30×35 , 35×15 , 15×5 , 5×10 , 10×20 , 20×25	(30,35,15,5,10,20,25)	15125	$((A_1(A_2A_3))((A_4A_5)A_6))$
5 matrices: 4×10 , 10×3 , 3×12 , 12×20 , 20×7	(4,10,3,12,20,7)	1344	$((A_1A_2)((A_3A_4)A_5))$

Check for the $4 \times 10 \dots 20 \times 7$ chain: $m[1,2]=120$, $m[3,4]=720$, $m[3,5]=1140$ (split after A_4), $m[1,5]=m[1,2]+m[3,5]+4 \cdot 3 \cdot 7=120+1140+84=1344$.

B.4 All-Pairs Shortest Path — Floyd-Warshall Algorithm

Definition

The all-pairs shortest path problem asks for the shortest distance between **every** pair of vertices of a weighted directed graph. Floyd-Warshall solves it with dynamic programming and **allows negative edge weights** (but not negative cycles).

Idea: Let $D^k[i,j]$ be the shortest $i \rightarrow j$ distance using only vertices $1 \dots k$ as intermediate stops. Going from D^{k-1} to D^k we ask: *is it shorter to go $i \rightarrow k \rightarrow j$ than the best known $i \rightarrow j$?*

Formula

$$D^k[i,j] = \min \{ D^{k-1}[i,j], D^{k-1}[i,k] + D^{k-1}[k,j] \}$$

D^0 = the weight (adjacency) matrix: 0 on the diagonal, edge weight if an edge exists, ∞ otherwise. D^n is the final answer.

```
Algorithm FloydWarshall(cost[1:n,1:n], n)
{
  for i := 1 to n do
    for j := 1 to n do
      D[i,j] := cost[i,j];           // D0
  for k := 1 to n do                // intermediate vertex
    for i := 1 to n do
      for j := 1 to n do
        D[i,j] := min( D[i,j], D[i,k] + D[k,j] );
  return D;
}
```

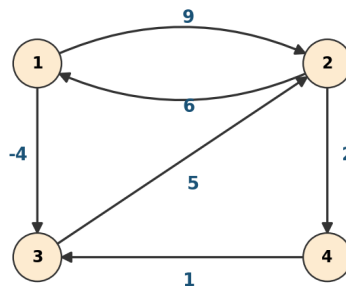
Formula

Time = $O(n^3)$ (three nested loops, each $O(1)$ work inside). Space = $O(n^2)$ (one matrix, updated in place).

Shortcut for exams

To get D^k from D^{k-1} : **copy row k, column k and the diagonal unchanged**; recompute every other cell as $\min(\text{old}, D[i,k] + D[k,j])$. That is only $(n-1)(n-2)$ cells per matrix.

Worked example (directed graph from your notes)



Edges: $1 \rightarrow 2$ (9), $2 \rightarrow 1$ (6), $1 \rightarrow 3$ (-4), $3 \rightarrow 2$ (5), $2 \rightarrow 4$ (2), $4 \rightarrow 3$ (1)

Matrices (rows/columns = vertices 1-4):

D^0

0	9	-4	∞
6	0	∞	2
∞	5	0	∞
∞	∞	1	0

D^3

D^1

0	9	-4	∞
6	0	2	2
∞	5	0	∞
∞	∞	1	0

D^4 (final)

D^2

0	9	-4	11
6	0	2	2
11	5	0	7
∞	∞	1	0

0	1	-4	3
6	0	2	2
11	5	0	7
12	6	1	0

0	1	-4	3
6	0	2	2
11	5	0	7
12	6	1	0

Sample calculations ($k=1$): $D^1[2,3] = \min\{D^0[2,3], D^0[2,1]+D^0[1,3]\} = \min\{\infty, 6+(-4)\} = 2$. ($k=2$): $D^2[1,4] = \min\{\infty, D^1[1,2]+D^1[2,4]\} = 9+2 = 11$. ($k=3$): $D^3[1,2] = \min\{9, D^2[1,3]+D^2[3,2]\} = -4+5 = 1$. ($k=4$): no cell improves, so $D^4 = D^3$.

Final answer D^4 gives all shortest distances, e.g. $1 \rightarrow 2 = 1$ (via 3), $1 \rightarrow 4 = 3$, $4 \rightarrow 1 = 12$, $3 \rightarrow 1 = 11$.

⚠ **Correction / Flag**

Kerala notes list “Bellman-Ford” as an All-Pairs algorithm. Bellman-Ford is a **single-source** algorithm (it handles negative edges). Only Floyd-Warshall (and Johnson’s) are all-pairs. Also, α in the slides = ∞ .

MODULE REVISION — Dynamic Programming

- **Definitions:** DP = store subproblem results to avoid recomputation. Principle of optimality = sub-solutions of an optimal solution are optimal. Properties: overlapping subproblems + optimal substructure.
- **Formulas:** $m[i,j] = \min_{i \leq k \leq j} \{m[i,k] + m[k+1,j] + p_{i-1}p_kp_j\}$ • $\text{cost}(p \times q \cdot q \times r) = pqr \cdot D^k[i,j] = \min\{D^{k-1}[i,j], D^{k-1}[i,k] + D^{k-1}[k,j]\}$.
- **Algorithms:** Matrix_Chain_Order $O(n^3)$ time, $O(n^2)$ space • Print_Optimal_Parens • FloydWarshall $O(n^3)$ time, $O(n^2)$ space.
- **Differences:** DP vs D&C; • DP vs Greedy • Floyd (all-pairs) vs Dijkstra (single-source, no negatives).

C. BACKTRACKING

C.1 Backtracking — Control Abstraction

Basic idea: Build the solution one component at a time. After each step, test whether the partial solution can still lead to a full answer. If not, **go back (backtrack)** to the last choice and try another. Think of solving a maze: at a dead end you step back to the last junction.

□ Definition

Backtracking is a depth-first search of the solution space (state space tree) that builds the solution vector $(x_1, x_2, \dots, x_n)$ component by component and uses **bounding functions** to abandon partial vectors that cannot lead to a solution.

Key terms

Term	Meaning
Solution vector	n-tuple $(x_1 \dots x_n)$, each x_i chosen from a finite set S_i
Explicit constraints	Rules limiting each x_i to a set (e.g. $x_i \in \{1..n\}$). They define the solution space
Implicit constraints	Rules saying how the x_i must relate to each other (e.g. no two queens in the same column/diagonal)
State space tree	Tree whose paths from root represent all candidate tuples; each node = a problem state
Solution state / Answer state	Path root→s is a tuple in the solution space / a tuple that satisfies all constraints
Live node	Generated, but children not all generated yet
E-node	Live node whose children are currently being generated
Dead node	Not expanded further (killed by bounding function, or all children done)
Bounding function B_k	Returns false when $(x_1 \dots x_k)$ can never be extended to an answer → node is killed

Why it is better than brute force: brute force tests all $m = m_1 \cdot m_2 \dots m_n$ tuples ($m_i = |S_i|$). If a partial vector $(x_1 \dots x_i)$ fails, backtracking discards **all** $m_{i+1} \dots m_n$ tuples below it without looking at them.

Recursive control abstraction

```

Algorithm Backtrack(k)
// x[1..k-1] already chosen. T(x[1..k-1]) = set of candidate values for x[k]
// B_k = bounding function. Initial call: Backtrack(1)
{
  for (each x[k] ∈ T(x[1], ..., x[k-1])) do
  {
    if ( B_k(x[1], x[2], ..., x[k]) ≠ 0 ) then
    {
      if ( x[1..k] is a path to an answer node ) then
        write( x[1:k] );
      if ( k < n ) then Backtrack(k+1);
    }
  }
}

```

Iterative version (same logic, explicit k)

```

Algorithm IBacktrack(n)
{
  k := 1;
  while (k ≠ 0) do
  {
    if (there remains an untried x[k] ∈ T(x[1..k-1]) and B_k(x[1..k]) is true) then
    {
      if (x[1..k] is a path to an answer node) then write(x[1:k]);
      k := k + 1;           // go deeper
    }
    else k := k - 1;       // backtrack
  }
}

```

Depth-first node generation: as soon as a child C of the E-node R is generated, C becomes the new E-node; R becomes the E-node again after C's subtree is fully explored. **Backtracking = depth-first generation + bounding functions.** Applications: N-Queens, sum of subsets, graph colouring, Hamiltonian cycles, 0/1 knapsack, maze solving.

C.2 N-Queens Problem

Definition

Place n queens on an $n \times n$ chessboard so that no two queens attack each other, i.e. no two are in the same row, column, or diagonal.

- $n = 1$ has a trivial solution; $n = 2$ and $n = 3$ have **no** solution; $n = 4$ has 2 solutions; $n = 8$ has 92.
- **Row trick:** since each row holds exactly one queen, queen i is placed in row i . A solution is the tuple $(x_1, \dots, x_n)$ where $x_i = \text{column of queen } i$.
- **Explicit constraint:** $S_i = \{1, \dots, n\} \rightarrow$ solution space n^n tuples.
- **Implicit constraints:** (1) all x_i distinct (different columns) $\rightarrow n!$ tuples; (2) no two queens on the same diagonal.

Diagonal test (important derivation)

Queens at (i, j) and (k, l) share a "****" diagonal if $i - j = k - l$, and a "**/**" diagonal if $i + j = k + l$. Rewriting: the first gives $j - l = i - k$, the second gives $j - l = k - i$.

Formula

Two queens are on the same diagonal $\Leftrightarrow |j - l| = |i - k|$
(difference of columns = difference of rows, in absolute value)

Algorithm

```

Algorithm NQueens(k, n)           // initial call: NQueens(1, n)
// prints all placements of n non-attacking queens; x[] is global
{
    for i := 1 to n do
    {
        if Place(k, i) then
        {
            x[k] := i;
            if (k = n) then write( x[1:n] );
            else NQueens(k+1, n);
        }
    }
}

Algorithm Place(k, i)
// true if queen k can go in column i (queens 1..k-1 already placed)
{
    for j := 1 to k-1 do
        if ( x[j] = i or Abs(x[j] - i) = Abs(j - k) ) then
            return false;           // same column or same diagonal
    return true;
}

```

- **Place(k,i)** is the bounding function; it takes $O(k)$ time. NQueens has worst-case time **$O(n!)$** (far less in practice because of pruning).
- Line-by-line: the loop over i tries each column for row k ; Place rejects unsafe columns; if the last row is filled ($k = n$) a solution is printed; otherwise we recurse to the next row. Returning from the recursion is the "backtrack".

Working of 4-Queens (step by step)

1. Place Q1 at (1,1).
2. Q2: columns 1 and 2 are attacked (same column / diagonal), so Q2 goes to (2,3).
3. Q3: every column in row 3 is attacked $\rightarrow$ dead end. **Backtrack**, move Q2 to (2,4).
4. Q3 can go to (3,2), but then Q4 has no safe column $\rightarrow$ dead end. Backtrack all the way past Q2 and Q3 to Q1.
5. Move Q1 to (1,2). Q2 $\rightarrow$ (2,4), Q3 $\rightarrow$ (3,1), Q4 $\rightarrow$ (4,3). All 4 placed: **solution (2, 4, 1, 3)**.

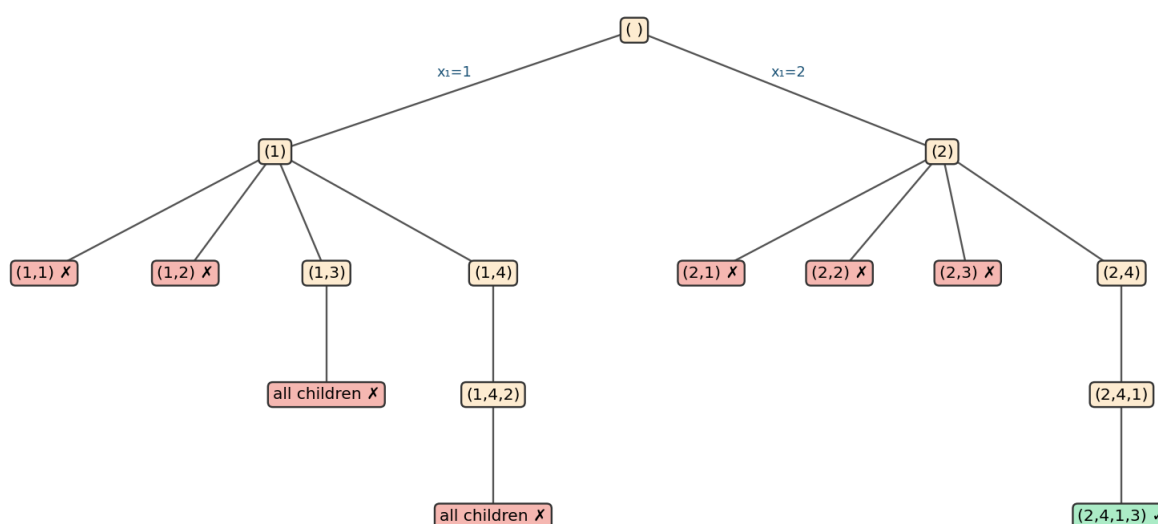
6. Continuing the search yields the second (mirror-image) solution **(3, 1, 4, 2)**.

	1	2	3	4
1		Q1		
2				Q2
3	Q3			
4			Q4	

	1	2	3	4
1			Q1	
2	Q2			
3				Q3
4		Q4		

The two solutions of 4-Queens (row = queen number, column = x_i)

State space tree (pruned) for 4-Queens



X = killed by bounding function (same column or same diagonal). Branches $x_1=3$ and $x_1=4$ are explored the same way: $x_1=3$ gives (3,1,4,2); $x_1=4$ is the mirror image of $x_1=1$ and dies.

Backtracking tree: dead nodes are killed immediately, so most of the $4^4 = 256$ (or $4! = 24$) leaves are never generated

8-Queens — one solution: (4, 6, 8, 2, 7, 1, 3, 5), i.e. Q1 in column 4 of row 1, Q2 in column 6 of row 2, ... Q8 in column 5 of row 8 (verified: all columns distinct, no equal diagonals).

△ Correction / Flag

In your lecture notes the state-space-tree figure (with nodes numbered 1–65) is the tree *without* the diagonal constraint (24 leaves = $4!$). Write which constraint your tree uses. The figure that shows nodes B (bounded/killed) is the backtracking version.

MODULE REVISION — Backtracking

- **Definitions:** Backtracking = DFS of state space tree with bounding functions. Explicit vs implicit constraints. Live node / E-node / dead node.
- **Formulas:** Same diagonal $\Leftrightarrow |j-i| = |k-i|$. Solution space $n^n \rightarrow n!$ after the distinct-column rule. Brute-force candidates $m = m_1 m_2 \dots m_n$.
- **Algorithms:** Backtrack(k) (recursive) • IBacktrack(n) • NQueens(k,n) • Place(k,i) $O(k)$.
- **Facts:** 4-Queens solutions (2,4,1,3), (3,1,4,2); 8-Queens has 92 solutions; none exist for $n = 2, 3$.

D. SYLLABUS COVERAGE CHECK

Syllabus topic	Covered	In your uploads?	Notes
Greedy – control abstraction	✓	No	Written from standard knowledge (A.1)
Fractional knapsack	✓	No	Algorithm + worked example, $O(n \log n)$
Kruskal's MST + analysis	✓	No	Trace + $O(E \log V)$
Prim's MST + analysis	✓	No	Trace + $O(V^2)/O(E \log V)$
Dijkstra + analysis	✓	No	Table trace + $O(V^2)/O(E \log V)$
DP – control abstraction	✓	Yes	4 steps, memoization added
Optimality principle	✓	Yes	Merged from both sources
Matrix chain multiplication + analysis	✓	Yes	3 examples; $O(n^3)$ verified by code
Floyd-Warshall + analysis	✓	Yes	Full D^0 - D^4 trace verified
Backtracking – control abstraction	✓	Yes	Recursive + iterative
N-Queens problem + algorithm	✓	Yes	4-queens, 8-queens, Place()
Excluded (not in syllabus)	—	Yes	Divide & Conquer slides, Branch & Bound, TSP

Quality check done

All numerical examples (knapsack, Kruskal, Prim, Dijkstra, three matrix-chain problems, Floyd-Warshall D^0 - D^4 , 8-queens) were recomputed by program and match. Errors found in the source notes are flagged in red boxes ($\alpha = \infty$ notation, Bellman-Ford as all-pairs, D&C-vs-DP; generalisation, a subscript typo).

E. LAST-MINUTE REVISION

Must-know definitions

- **Greedy:** choose the locally best option at each step, never revise.
- **MST:** spanning tree ($V-1$ edges) with minimum total weight.
- **Dynamic programming:** solve overlapping subproblems once, store, reuse.
- **Principle of optimality:** in an optimal sequence of decisions, the remaining decisions are optimal for the state after the first decision.
- **Backtracking:** depth-first search of the state space tree with bounding functions; go back on a dead end.
- **Bounding function:** test that kills a node whose partial vector cannot reach an answer.

Must-know formulas

- Knapsack ratio = p_i/w_i (sort descending).
- Relax: $d[v] = \min(d[v], d[u] + w(u,v))$.
- Matrix cost: $p \times q \cdot q \times r = pqr$.
- $m[i,j] = \min_{i \leq k < j} \{ m[i,k] + m[k+1,j] + p_{i-1}p_kp_j \}$.
- $D^k[i,j] = \min \{ D^{k-1}[i,j], D^{k-1}[i,k] + D^{k-1}[k,j] \}$.
- Queens attack diagonally $\Leftrightarrow |j-i| = |i-k|$.

Complexity table

Algorithm	Time	Space / Remark
Fractional knapsack	$O(n \log n)$	sorting dominates
Kruskal	$O(E \log V)$	sort + union-find
Prim	$O(V^2) / O(E \log V)$	matrix / heap
Dijkstra	$O(V^2) / O(E \log V)$	no negative edges
Matrix chain	$O(n^3)$	$O(n^2)$ tables m, s
Floyd-Warshall	$O(n^3)$	$O(n^2)$, negative edges OK
Place(k, i)	$O(k)$	—
N-Queens	$O(n!)$ worst case	pruned in practice

Important diagrams to practise drawing

- 4-Queens state space (pruned) tree and the two solution boards.
- Floyd-Warshall matrices $D^0 \dots D^n$.
- Matrix-chain tables m and s .
- MST of a small graph (edge-by-edge) and Dijkstra table.

Important comparisons

- Kruskal vs Prim • Greedy vs DP • D&C; vs DP • Dijkstra vs Floyd • Fractional vs 0/1 knapsack • Explicit vs implicit constraints • Backtracking vs brute force.

Common mistakes

- Fractional knapsack: sorting by profit or by weight instead of **profit/weight**.
- Kruskal: forgetting the cycle check, or stopping after the wrong number of edges (must be $V-1$).
- Dijkstra with negative edges; forgetting that a final vertex is never updated again.
- Matrix chain: using p_i instead of $p_{i-1} \cdot p_k \cdot p_j$; writing the dimension list wrongly (n matrices $\rightarrow n+1$ numbers).
- Floyd: changing row k / column k (they stay unchanged in D^k); forgetting diagonal = 0.
- N-Queens: checking only columns and forgetting the diagonal test; saying Place is $O(n)$ (it is $O(k)$).
- Calling the 4-queens tree "24 leaves" without saying it ignores diagonals.

Topics needing extra attention

- **Numerical problems:** Matrix chain (always asked with m and s tables), Floyd-Warshall (3-4 vertex graph), Kruskal/Prim/Dijkstra tables.
- **Writing algorithms:** Greedy abstraction, Backtrack(k), NQueens + Place, Matrix_Chain_Order.
- **Proof-flavoured answers:** principle of optimality, derivation of $|j-l| = |i-k|$.

F. QUESTION BANK

No previous papers were provided, so the mark split below is a typical KTU style (3-mark short, 7-mark, 14-mark). Adjust to your actual pattern. Priority tags are based on emphasis in your notes.

Greedy Strategy

- **[Very Important]** Explain the control abstraction of the greedy method. (3)
- **[Very Important]** Solve the fractional knapsack problem: $n=3$, $M=50$, $p=(60,100,120)$, $w=(10,20,30)$. Write the algorithm. (7)
- **[Very Important]** Explain Kruskal's algorithm with an example and analyse its complexity. (7/14)
- **[Very Important]** Explain Prim's algorithm with an example. Compare with Kruskal. (7/14)
- **[Very Important]** Write Dijkstra's algorithm and find the shortest paths from a given source in a weighted graph. (14)
- **[Important]** Why does Dijkstra's algorithm fail with negative edge weights? (3)
- **[Important]** Define spanning tree and minimum cost spanning tree. (3)
- **[Important]** Why is greedy optimal for fractional but not 0/1 knapsack? (3/7)
- **[Practice]** Differentiate greedy method and dynamic programming. (3)

Dynamic Programming

- **[Very Important]** Explain the control abstraction of dynamic programming. (3)
- **[Very Important]** State the principle of optimality. Give an example. (3)
- **[Very Important]** Using dynamic programming, find the optimal parenthesization of $A_1 \dots A_4$ with dimensions 5×4 , 4×6 , 6×2 , 2×7 . (7/14)
- **[Very Important]** Find the optimal parenthesization for the chain 30×35 , 35×15 , 15×5 , 5×10 , 10×20 , 20×25 . (14)
- **[Very Important]** Find the optimal parenthesization for 4×10 , 10×3 , 3×12 , 12×20 , 20×7 . (7/14)
- **[Very Important]** Write the Floyd-Warshall algorithm, analyse its time complexity and solve for a given 4-vertex directed graph. (14)
- **[Important]** Write the algorithm for matrix chain order and show that its complexity is $O(n^3)$. (7)
- **[Important]** What are overlapping subproblems and optimal substructure? (7)
- **[Important]** Compare divide-and-conquer and dynamic programming. (3/7)
- **[Practice]** Differentiate Floyd-Warshall and Dijkstra. (3)
- **[Practice]** What is memoization? (2)

Backtracking

- **[Very Important]** Explain the control abstraction of backtracking. (3/7)
- **[Very Important]** Explain the N-Queens problem. Write the NQueens and Place algorithms. (7)
- **[Very Important]** Draw the state space tree of the 4-Queens problem and show the steps to find all solutions using backtracking. (14)
- **[Important]** Differentiate explicit and implicit constraints with examples. (3)
- **[Important]** Derive the condition for two queens to be on the same diagonal. (3)
- **[Important]** Define live node, E-node, dead node, bounding function. (3)
- **[Practice]** Give a solution of the 8-Queens problem. (3)
- **[Practice]** What is the time complexity of Place(k,i) and NQueens? (2)
- **[Practice]** Differentiate backtracking and brute force. (3)

Numerical practice set (do these on paper)

- Fractional knapsack: $n=4$, $M=40$, $p=(280,100,120,120)$, $w=(40,10,20,24)$. (Check: ratios 7, 10, 6, 5 $\rightarrow$ take item 2, item 1 fraction 30/40; profit $100+210 = 310$.)
- Matrix chain: $p = (10, 20, 30, 40, 30)$. (Answer: 30000, $((A_1 A_2) A_3) A_4$.)
- Floyd-Warshall on your notes' graph and on one you draw yourself with a negative edge.

- Kruskal and Prim on the graph in Section A; confirm both give 13. Run Dijkstra from F instead of A.

Programming / diagram questions

- **[Very Important]** Write pseudocode for NQueens and Place; trace for $n = 4$. (7)
- **[Important]** Write pseudocode for Kruskal with union-find. (7)
- **[Practice]** Draw the state space tree for $n = 3$ queens and show that no solution exists. (7)